

MODULE 4

System Design & Architecture

Renaissance Developer Academy — Detailed Lesson Plan

Week 4 of 10 | 5 Days | Full-Time Intensive
Badge Earned: Systems Architect

Prerequisite: Module 3 — Full-Stack Builder
DRAFT — March 2026

Module Overview

Module 3 proved you can build fast. Module 4 asks a harder question: **can you design well?** Speed without architecture creates systems that are expensive to change, fragile under load, and impossible to reason about. This week shifts from building to **designing** — learning to make trade-offs explicit, document decisions rigorously, and evaluate architectural options before committing to code.

The week pairs **system design thinking** (progressive complexity from single-service to distributed systems) with **security-first architecture analysis** using NanoClaw as the hands-on case study. NanoClaw — the container-isolated, security-focused agent from the OpenClaw ecosystem — represents the next step up from Module 1–2’s Nanobot: same concept, radically different architectural choices.

Connection to the JD: This module maps directly to Architecture & Design (“design complex multi-component systems end-to-end, evaluate architectural options”), Code Quality & Review (“establish and enforce quality standards”), and the Systems Thinking behavior (“shape systems design practices, create clarity from complexity”). The ADR practice introduced this week becomes a permanent habit.

Learning Objectives

1. Design multi-component systems with documented trade-off analysis using progressive complexity (single-service → multi-service → distributed)
2. Write Architecture Decision Records (ADRs) using the Context → Decision → Consequences template for every significant design choice
3. Master Kiro’s spec-driven development workflow at architecture scale: requirements → design → tasks → implementation
4. Deploy, analyze, and critique NanoClaw’s container-isolation architecture as a security-first design case study
5. Evaluate architectural options by articulating trade-offs between competing qualities (performance vs. simplicity, consistency vs. availability, security vs. usability)
6. Lead a system design review: present, defend, and iterate on architectural decisions based on peer critique

Pre-Work (Assigned End of Module 3)

Due before Day 1. Estimated time: 3–4 hours.

1. **Reading:** Read Chapters 1–2 of “Designing Data-Intensive Applications” by Martin Kleppmann (Reliable, Scalable, and Maintainable Applications; Data Models and Query Languages). Upload to your NotebookLM notebook and generate an Audio Overview.
2. **NanoClaw Reconnaissance:** Clone the NanoClaw repository from GitHub. Read the README and skim the codebase (~5 files, ~4000 lines). Don’t try to run it yet — just

understand the project's goals and architecture at a high level. Note: how is it different from Nanobot?

3. **ADR Template:** Read the ADR format at adr.github.io. Familiarize yourself with the structure: Title, Status, Context, Decision, Consequences. Write a practice ADR for one technology choice from your Module 3 application (e.g., "Why we chose PostgreSQL over MongoDB").
4. **Reflection:** Review your Module 3 architecture document. Identify 3 design decisions you made implicitly (without conscious deliberation). What were the alternatives you didn't consider? Bring these to Day 1.

Week-at-a-Glance

Day	Theme	Primary Tools	Key Output
Day 1	System Design Thinking & ADR Fundamentals	Whiteboard, Kiro, NotebookLM	2 system designs, 2 ADRs, design journal
Day 2	Distributed Systems & Trade-Off Analysis	Kiro, Claude Code, whiteboard	Distributed system design, trade-off matrix
Day 3	NanoClaw: Security-First Architecture	NanoClaw, Docker, Claude Code	NanoClaw deployed, security analysis report
Day 4	Spec-Driven Architecture Build	Kiro, Claude Code, GitHub Actions	Spec-driven app built entirely through Kiro workflow
Day 5	Architecture Reviews, Reflection & Calibration	All tools, NotebookLM	Design review presentations, development map v4

Day 1: System Design Thinking & ADR Fundamentals

Theme: Learn to think in systems, make trade-offs explicit, and document decisions before they become invisible assumptions

Goal: Every student has designed 2 systems of increasing complexity, written 2 ADRs, and can articulate the difference between making a decision and documenting a decision.

Schedule

09:00	Week 4 Opening: From Building Fast to Designing Well	30 min
	- The shift: Module 3 asked “can you build it?” Module 4 asks “should you build it THIS way?” - The Renaissance Developer’s Systems Thinking behavior: “create clarity from complexity, bridge technical systems with business processes” - Reflection share: the 3 implicit design decisions from your Module 3 app (from pre-work) — quick round-robin - Key insight: every implicit decision is a hidden risk. Architecture is the practice of making decisions explicit. - This week’s mantra: “Trade-offs, not best practices.” There is no single correct design — only designs that optimize for different qualities.	
09:30	System Design Fundamentals (Concept Session)	75 min
	- The system design thinking framework (not a checklist, but a way of reasoning): — Step 1: Clarify requirements — functional, non-functional, constraints, and assumptions — Step 2: Define the interface — what does the outside world see? APIs, events, data contracts — Step 3: Sketch the high-level design — components, data flow, boundaries — Step 4: Deep-dive critical components — where are the hard problems? — Step 5: Evaluate trade-offs — what did you optimize for? What did you sacrifice? - Three qualities that are always in tension: — Performance vs. Simplicity: the fastest solution is often the most complex — Consistency vs. Availability: the CAP theorem is not academic — it drives real decisions — Security vs. Usability: every security measure adds friction — where is the right balance? - Live design walkthrough: instructor designs a URL shortener on the whiteboard using the 5-step framework — Students challenge each decision: “Why a relational DB? Why not a key-value store? What about cache invalidation?” — The instructor deliberately makes sub-optimal choices to show how trade-off discussion works	

10:45	Break	15 min
	• Questions	
11:00	Architecture Decision Records: Theory & Practice	45 min
	• What is an ADR and why it matters: — A lightweight document capturing: Context (what's the situation?), Decision (what did we choose?), Consequences (what are the trade-offs?) • — Status: Proposed → Accepted → Deprecated → Superseded • — ADRs are immutable — you don't edit old ADRs, you supersede them with new ones • — This creates a decision log: future engineers can understand WHY the system looks the way it does • Good ADR vs. bad ADR: — Bad: "We chose React." (no context, no alternatives, no consequences) — Good: "Given our team's experience with React, the need for SSR, and our timeline, we chose Next.js over Remix and Astro. We accept the trade-off of Next.js's opinionated routing for its mature SSR story." • Connection to the Precision mindset: ADRs are spec-driven thinking applied to architecture decisions • EXERCISE 1: Write an ADR for a Module 3 decision you made implicitly — Use the template: Title, Status (Accepted), Context, Decision, Consequences — Include at least 2 alternatives you considered (or should have considered) — Be specific about what you gained and what you gave up	
11:45	Lunch	60 min
	• Optional: browse ADR examples at github.com/joelparkerhenderson/architecture-decision-record	
12:45	System Design Workshop: Problem 1 (Single-Service)	90 min
	• EXERCISE 2: Design a real-time notification system • Requirements given to students: — Support 100K active users sending/receiving notifications — Notifications can be: in-app, email, push, or SMS — Users can configure preferences (which channels, quiet hours, priority levels) — Notifications must be delivered within 5 seconds for in-app, 30 seconds for others — System must handle bursts (10x normal volume during announcements) • Design process (whiteboard or digital): — Phase 1 (15 min, solo): sketch your initial design using the 5-step framework — Phase 2 (20 min, pairs): present to your partner, they challenge every decision — Phase 3 (20 min, solo): revise based on feedback — Phase 4 (20 min): write the ADR for your most important design decision	

	— Phase 5 (15 min): class discussion — compare approaches, what trade-offs emerged? - Common design tension: should notifications be push-based (WebSocket) or pull-based (polling)? Why? - Common trap: students design for the average case, not the burst case — the requirements said 10x spikes
--	--

14:15	Break	15 min
	Stretch	

14:30	System Design Workshop: Problem 2 (Multi-Service)	90 min
	- EXERCISE 3: Design an e-commerce order processing pipeline - Requirements (deliberately more complex than Problem 1): — User places order → payment processed → inventory reserved → order confirmed → shipping initiated — Each step can fail independently — design for partial failure and recovery — Support 10K orders/hour with 99.9% success rate — Must be auditable: every state change logged for compliance — Must support multiple payment providers (Stripe, PayPal, bank transfer) - New concepts this problem introduces: — Service boundaries: where do you split? Payment service, inventory service, order service? — Event-driven vs. synchronous: should services call each other directly or communicate via events? — Saga pattern: how to handle distributed transactions when each service has its own database — Idempotency: what happens if the payment webhook fires twice? - Same design process as Problem 1 (solo → pair challenge → revise → ADR → class discussion) - Write a second ADR: your decision on synchronous vs. event-driven communication between services	

16:00	Design Journal & Day 1 Wrap-Up	30 min
	- EXERCISE 4: Start your System Design Journal — Create a design-journal/ directory in your portfolio repo — Commit both system designs (Problem 1 and 2) as Markdown files with diagrams — Commit both ADRs in an adrs/ directory — Write a short reflection: what was the hardest trade-off you encountered today? - HOMEWORK: Read your partner's designs and ADRs. Leave 2 substantive comments as GitHub Issues. - HOMEWORK: Read DDIA Chapter 3 (Storage and Retrieval) — focus on the trade-offs between storage engines	

- | | |
|--|---|
| | <ul style="list-style-type: none">• Preview: tomorrow we tackle distributed systems and the hard problems |
|--|---|

Instructor Notes — Day 1: Day 1 is the conceptual pivot of the program — from “can I build it?” to “should I build it this way?” The URL shortener live design should model the behavior you want: thinking aloud, considering alternatives, being willing to say “I’m not sure which is better” openly. The pair challenge in Exercises 2 and 3 is critical — students must practice pushing back on each other’s designs respectfully. Coach toward “What happens when X fails?” and “Why this and not that?” rather than “That’s wrong.” The ADR quality varies wildly on the first attempt — the most common failure is writing an ADR that documents the choice but not the context or alternatives. Push students to articulate the road not taken.

Day 2: Distributed Systems & Trade-Off Analysis

Theme: Confront the hard problems of distributed systems: consistency, availability, partitioning, and the art of choosing what to sacrifice

Goal: Every student has designed a distributed system, built a trade-off analysis matrix, and can explain the CAP theorem, eventual consistency, and the saga pattern in practical (not academic) terms.

Schedule

09:00	Distributed Systems: The Hard Problems (Concept Session)	75 min
	- Why distributed systems are fundamentally different from single-service applications: — Network partitions are not edge cases — they are normal operating conditions — Clocks are unreliable: two servers may disagree on what time it is — Partial failure: component A succeeds while component B fails — now what? - The CAP theorem in practice (not theory): — Consistency: every read returns the most recent write — Availability: every request receives a response (even if stale) — Partition tolerance: the system continues operating despite network failures — The real choice: CP (consistent but sometimes unavailable) vs. AP (always available but sometimes stale) — Examples: banking (CP — consistency is non-negotiable) vs. social media feed (AP — stale is OK) - Eventual consistency: a practical middle ground — The system will converge to a consistent state, but reads may return stale data temporarily — When it works: shopping carts, social media likes, analytics dashboards — When it fails: bank balances, inventory counts for limited-stock items - Data partitioning strategies: sharding by user, by geography, by hash — Trade-offs: cross-partition queries become expensive, data hotspots, rebalancing complexity	
10:15	Break	15 min
	Questions	
10:30	System Design Workshop: Problem 3 (Distributed System)	105 min
	- EXERCISE 5: Design a collaborative document editor (like Google Docs, simplified) - Requirements:	

	— Support 50 concurrent editors on the same document — Changes appear within 200ms for all editors — Offline editing must be supported (sync when reconnected) — Complete edit history with undo/redo per user — Documents up to 10MB with rich text formatting This problem forces engagement with: — Conflict resolution: two users edit the same paragraph simultaneously — who wins? — Operational Transform (OT) vs. CRDT: two fundamentally different approaches to real-time collaboration — Consistency model: strong consistency kills performance at 50 concurrent editors; eventual consistency creates confusing UX — Offline-first architecture: local-first data that syncs when connectivity returns Design process: — Phase 1 (20 min, solo): initial design — Phase 2 (25 min, groups of 3–4): present and challenge each other's designs — Phase 3 (20 min, solo): revise with a focus on the hardest problem (conflict resolution) — Phase 4 (20 min): write an ADR for your conflict resolution strategy (OT vs. CRDT vs. last-writer-wins) — Phase 5 (20 min): group presentations — each group presents their strongest design to the class Instructor walks through a reference design after presentations, highlighting trade-offs students missed
--	--

12:15	Lunch	60 min
	Optional: DDIA Chapter 3 discussion group	

13:15	Trade-Off Analysis Framework (Workshop)	60 min
	- EXERCISE 6: Build a Trade-Off Analysis Matrix - A structured tool for comparing architectural options: — Columns: the qualities that matter (performance, scalability, simplicity, security, cost, time-to-market, team familiarity) — Rows: the architectural options being compared — Cells: score (1–5) + brief justification for each quality per option — Weights: not all qualities are equally important — which matter most for this specific context? - Apply the matrix to your Problem 3 design: — Compare OT vs. CRDT vs. last-writer-wins on the same matrix — Change the weights: what if latency matters more than consistency? What if offline support is the top priority? — Observe: changing weights changes the “best” option — this is why context always beats best practices - Key insight: the matrix is not a formula that produces the right answer. It is a communication tool that makes your reasoning transparent to others.	

	Commit the matrix template to your portfolio repo — you'll use it for all future architecture decisions
--	---

14:15	Break	15 min
	Stretch	

14:30	AI-Assisted System Design: When It Helps and When It Misleads	60 min
	- EXERCISE 7: Use AI tools for system design assistance — then critique the results - Use Claude Code CLI or Kiro to design the same collaborative editor system: — Prompt: “Design a system architecture for a collaborative document editor supporting 50 concurrent users with offline editing” — Evaluate the AI’s output against your own design: — Did the AI consider the same trade-offs you did? — Did it miss anything critical? (often: offline sync, conflict resolution details, failure modes) — Did it propose a design that sounds good but wouldn’t work at the specified scale? — Did it default to “best practice” answers instead of context-specific trade-offs? - Discussion: AI is useful for exploring options and generating initial architectures, but it consistently fails at: — Understanding your specific constraints and non-functional requirements — Reasoning about failure modes and partial failures — Making genuine trade-offs (AI tends to say “use both” instead of choosing) — Adapting patterns to unusual contexts - Write an entry in your design journal: “When I would and wouldn’t use AI for system design”	

15:30	Kiro Spec-Driven Architecture at Scale	45 min
	- EXERCISE 8: Use Kiro to generate a full spec for one of your Problem 1–3 designs — Feed Kiro your system design description and requirements — Let it generate requirements.md, design.md, and tasks.md — Evaluate: how well does Kiro’s spec capture the trade-offs you identified? — Enhance: add the non-functional requirements and trade-offs Kiro missed — Use EARS notation for requirements where applicable (Event, Action, Response, State) - This prepares for Day 4 when you build an entire application from Kiro specs	

16:15	Day 2 Wrap-Up	30 min
	Commit all Day 2 work: Problem 3 design, ADR, trade-off matrix, AI critique, Kiro spec	

	• HOMEWORK: Complete your Kiro spec enhancement. Read the NanoClaw README and Dockerfile in detail. • Preview: tomorrow we deploy NanoClaw and conduct a security architecture analysis
--	---

Instructor Notes — Day 2: Day 2 is the intellectually densest day of Module 4. The collaborative editor problem is deliberately chosen because it has no clean solution — every approach involves real sacrifice. Students who default to “just use CRDTs” should be asked to explain how CRDTs handle formatting conflicts in rich text (they don’t handle this elegantly). The trade-off matrix exercise transforms hand-wavy reasoning into structured analysis. The AI-assisted design exercise (Exercise 7) often produces the week’s best teaching moment: students see AI generate plausible-sounding architectures that collapse under scrutiny. This cements the lesson that architecture is a human judgment skill, not a generation skill.

Day 3: NanoClaw — Security-First Architecture

Theme: Study, deploy, and critically analyze NanoClaw as a case study in security-first architectural design

Goal: Every student has NanoClaw deployed in containers, has completed a structured security architecture analysis comparing NanoClaw to Nanobot and OpenClaw, and can articulate why container isolation matters for autonomous agents.

Schedule

09:00	Security-First Architecture: Principles & Patterns (Concept Session)	45 min
	- The security-first design philosophy: — Security is not a feature you add later — it is an architectural constraint that shapes everything — “Shift left”: security decisions are cheapest at design time, most expensive in production — Defense in depth: multiple layers, any one of which can fail without catastrophe - Container isolation as an architectural pattern: — Process isolation: the agent can’t access the host system even if compromised — Filesystem isolation: the agent only sees the data you explicitly mount — Network isolation: the agent’s network access is controlled by container policy — Resource limits: CPU, memory, disk capped — a runaway agent can’t consume the entire machine - NanoClaw’s origin story: built specifically because OpenClaw’s 430K-line codebase is unauditable — NanoClaw: ~5 files, ~4000 lines, built on Claude Code, containers by default — OpenClaw: 430K+ lines, binds to 0.0.0.0 by default, 135K+ instances found exposed on internet — Architectural insight: sometimes the most secure design is the one with the least code - Connection to the Architect competency layer: Architecture & Design + Code Quality + SRE	
09:45	NanoClaw Deep Dive: Code & Architecture Analysis	75 min
	- EXERCISE 9: Read and analyze the NanoClaw codebase - NanoClaw’s architecture (compare with Nanobot from Modules 1–2): — Core engine: how does message routing work? How are tools invoked? — Container model: how does NanoClaw create and manage isolated containers? — Skill system: how are skills loaded? How does this differ from ClawHub’s approach? — Claude Code integration: NanoClaw is built ON Claude Code, not as a separate agent — why?	

	• Comparison exercise (document in your design journal): • — Nanobot (Module 1): minimal, no isolation, single-process, local memory files • — NanoClaw (this week): container-isolated, security-focused, built on Claude Code • — OpenClaw (Module 8): full-featured, plugin ecosystem, significant attack surface • For each: draw the trust boundary diagram. Where does trusted code end and untrusted input begin? • Write the architectural comparison as a structured analysis: what does each optimize for? What does each sacrifice?
--	---

11:00	Break	15 min
	• Troubleshoot any Docker issues before deployment	

11:15	Hands-On: Deploy NanoClaw in Containers	75 min
	• EXERCISE 10: Deploy and configure NanoClaw locally • — Build the NanoClaw Docker image from the provided Dockerfile • — Configure: connect to an LLM provider (Claude API or local model) • — Configure: connect to a messaging platform (reuse your Module 1 Telegram/Discord setup) • — Run in container: verify the agent responds to messages • — Test container isolation: • — Try to access host files from within the container — verify it's blocked • — Try to access the host network — verify it's restricted • — Set resource limits (CPU, memory) and verify the agent respects them • — Run multiple NanoClaw instances: each in a separate container with different configurations • — Verify: instances are truly isolated from each other (no shared state, no cross-contamination) • Compare experience: how does running NanoClaw feel vs. running Nanobot directly on your machine?	

12:30	Lunch	60 min
	• Optional: container security deep-dive discussion	

13:30	Security Architecture Analysis Workshop	90 min
	• EXERCISE 11: Conduct a structured security architecture analysis of NanoClaw • Using the threat modeling framework from Module 1 Day 3, expanded to architecture scale: • Part 1 — Attack Surface Mapping (30 min): • — List every entry point: user messages, API keys, LLM responses, skill code, container orchestration	

	— For each entry point: what can an attacker send? What's the worst outcome? — Prompt injection analysis: what happens if a malicious message tries to override NanoClaw's instructions? - Part 2 — Architecture-Level Security Assessment (30 min): — Evaluate NanoClaw's security claims against actual implementation: — "Container isolation": is the container configured securely? Non-root user? No host mounts? — "Auditable codebase": is it actually auditable? Can you understand the security model from the code? — "Skill vetting": how does NanoClaw handle skills vs. OpenClaw's ClawHub (17% malicious)? — Compare: what security guarantees does NanoClaw provide that OpenClaw doesn't? - Part 3 — Recommendations (30 min): — Write 5–10 security recommendations for anyone deploying NanoClaw in production — Classify each: Critical / High / Medium / Low severity — For each recommendation: the threat, the fix, and the trade-off (what does the fix cost in usability or performance?) - Compile into a Security Analysis Report (committed to your design journal)
--	---

15:00	Break	15 min
	Stretch	

15:15	ADR: Container Isolation as an Architectural Decision	45 min
	- EXERCISE 12: Write an ADR for the container isolation decision — Context: you're deploying an autonomous AI agent that can execute arbitrary code — Options considered: (a) direct host execution (Nanobot approach), (b) container isolation (NanoClaw), (c) sandboxed VM (heavier isolation), (d) serverless functions (ephemeral execution) — Decision: argue for one based on your analysis — Consequences: what you gain and what you lose vs. each alternative - Use your trade-off analysis matrix from Day 2 to structure the comparison - This ADR should be your strongest yet — informed by actual deployment experience, not just theory	

16:00	Day 3 Wrap-Up	30 min
	- Commit all work: NanoClaw config, architecture comparison, security analysis, container isolation ADR - Quick share: most surprising security finding from today's analysis - HOMEWORK: Refine your security analysis report based on peer feedback - Preview: tomorrow we build a complete application using Kiro's spec-driven workflow from end to end	

Instructor Notes — Day 3: Day 3 is the OpenClaw ecosystem's second major appearance (after Nanobot in Modules 1–2) and the security theme connects back to Module 2's agent security workshop. The deployment exercise (Exercise 10) should be relatively smooth since students have Docker experience from Module 3 — the interesting part is testing isolation, not installation. The security analysis (Exercise 11) is the day's centerpiece. Push students beyond surface-level findings toward architecture-level insights. The best analyses will connect NanoClaw's design choices to the security principles from the morning concept session. Students who say "it's secure because it uses containers" should be challenged: containers are not security boundaries in all threat models.

Day 4: Spec-Driven Architecture Build

Theme: Build an entire application driven by Kiro's specification workflow — requirements before code, design before implementation

Goal: Every student has built a working application entirely through Kiro's spec-driven workflow, has experienced the discipline of not writing code until specs are approved, and can compare spec-driven development to their Module 3 build-first approach.

Schedule

09:00	Spec-Driven Development at Architecture Scale (Concept Session)	30 min
	- Recap: Kiro's workflow (requirements.md → design.md → tasks.md → code) - Why spec-driven matters at architecture scale: — Requirements are the contract between the builder and the stakeholder — Design documents are the architectural blueprint — ADRs capture the decisions, design.md captures the structure — Tasks are the execution plan — if you can't break a design into tasks, you don't understand it yet - EARS notation refresher: Event-driven, Action-based, Response-oriented, State-dependent requirements - Today's challenge: build a new application entirely through Kiro's spec workflow. NO CODE until specs are reviewed.	
09:30	Spec Phase: Requirements & Design	90 min
	- EXERCISE 13: Generate and refine a complete specification through Kiro - Choose a new application (different from Module 3) — suggested options: — Option A: A team retrospective tool (create retros, vote on items, generate action items, track follow-ups) — Option B: An API monitoring dashboard (register endpoints, schedule health checks, alert on failures, visualize uptime) — Option C: A knowledge base/wiki (Markdown-based, full-text search, version history, team collaboration) — Option D: Propose your own — must have 3+ entities and at least 1 non-trivial workflow - Use Kiro's spec generation: — Provide a 2–3 paragraph description with user personas and key use cases — Let Kiro generate requirements.md — review for completeness, add missing non-functional requirements — Let Kiro generate design.md — review architecture, challenge component boundaries, verify data model — Let Kiro generate tasks.md — review for realistic scope, re-order for dependencies	

	• Peer review gate: your accountability partner must review and approve your specs before you write any code • — They should challenge: “Are the requirements testable? Is the design consistent? Are the tasks achievable today?” • Commit all spec documents. This is the last time you touch requirements — code must match specs.
--	---

11:00	Break	15 min
	• Spec review conversations	

11:15	Build Phase: Spec-Driven Implementation	75 min
	• EXERCISE 14: Build the application following the spec and task breakdown • — Work through tasks.md in order — each task should produce a testable increment • — Let Kiro generate code for each task, then verify against the design.md • — Use Agent Hooks: auto-test on save, auto-lint on save • — After each task: check — does the implementation match the requirements? • Discipline: if you discover a spec gap during implementation, update the spec FIRST, then update the code • — This feels slow but prevents “spec drift” where docs and code diverge • Commit after each task with a message referencing the task ID from tasks.md	

12:30	Lunch	60 min
	• Keep building if you're in flow	

13:30	Build Phase: Continued Implementation + CI/CD	90 min
	• EXERCISE 14 (continued): Complete the remaining tasks • — At minimum: all tasks from the spec should be started; high-priority tasks must be complete • — Add CI pipeline: lint → test → build (reuse your Module 3 workflow patterns) • — Containerize: write a Dockerfile (reuse multi-stage pattern from Module 3) • — Deploy to your cloud platform (reuse your deployment workflow) • 14:30 milestone: “How many tasks are complete? Is CI green? Is it deployed?” • If time permits: use Kiro’s property-based testing to generate tests from your requirements • — Property-based tests extract properties from specs and test with random inputs — more thorough than example-based tests	

15:00	Break	15 min
	• Stretch	

15:15	Spec-Driven vs. Build-First: Comparative Analysis	45 min
	• EXERCISE 15: Compare this week's spec-driven approach to Module 3's build-first approach • Write a structured comparison (committed to your design journal): • — Speed: which approach felt faster? At what cost? • — Quality: where were there fewer bugs? Why? • — Completeness: which approach produced more complete documentation? • — Changeability: if you needed to pivot the design, which codebase would be easier to change? • — Team scalability: which approach would work better with 5 engineers instead of 1? • — AI tool effectiveness: did AI produce better code with specs as context? How much better? • Group discussion: when should you use spec-driven development? When is build-first better? • Key insight: spec-driven is slower for solo prototyping but dramatically better for team delivery and long-lived systems	
16:00	Day 4 Wrap-Up	30 min
	• Commit all work: spec docs, implementation, CI/CD, deployment, comparison analysis • HOMEWORK: Complete any remaining tasks from the spec. Ensure deployment is stable. • HOMEWORK: Prepare for tomorrow's architecture review: organize your design journal, polish ADRs • Preview: tomorrow — architecture review presentations and the module's capstone assessment	

Instructor Notes — Day 4: Day 4's spec-driven build is a disciplined exercise that some students will find frustratingly slow compared to Module 3's pace. That friction is the lesson. Students who "cheat" by writing code without specs should be gently redirected — the whole point is experiencing the discipline. The peer review gate before coding is essential; if partners rubber-stamp specs, the exercise loses its value. The comparative analysis (Exercise 15) is the day's most important output intellectually. Push students toward nuanced answers, not "spec-driven is better." The right answer is context-dependent — which connects back to the trade-off thinking from Days 1–2.

Day 5: Architecture Reviews, Reflection & Calibration

Theme: Present, defend, and iterate on architectural decisions; reflect on the growth from builder to architect

Goal: Every student has presented their architecture to peers, received and addressed structured feedback, and updated their Personal Development Map to reflect the shift from Layer 1 (Technical Foundation) to Layer 2 (Architect).

Schedule

09:00	Architecture Review Presentations	120 min
	- EXERCISE 16: Present your system design portfolio to the cohort - Each student: 7 minutes (strictly timed) presenting the week's best architectural work - Presentation structure: — Design Context (1 min): the problem, the constraints, what you optimized for — Architecture Walkthrough (2 min): the design with key components and data flow — Key ADR (1 min): your strongest architectural decision with the reasoning — NanoClaw Security Insight (1 min): the most important finding from your security analysis — Spec-Driven Reflection (1 min): what you learned from spec-driven vs. build-first — What I'd change (1 min): if you redesigned from scratch, what one thing would be different? - After each presentation: 3 minutes of structured peer feedback: — One architectural strength (what trade-off was well-reasoned) — One challenge ("What happens if X fails?" or "Did you consider Y?") - This is practice for the JD's requirement: "guide technical decisions for your area, mentor engineers on architecture"	
11:00	Break	15 min
	Decompress after presentations	
11:15	Architecture Review: Written Feedback	45 min
	- EXERCISE 17: Provide written architecture reviews for 2 peers - Each review includes: — Review of their design journal: are the system designs well-reasoned? Are ADRs clear and complete? — Review of their NanoClaw security analysis: are findings actionable? Are severity ratings justified?	

	— Review of their spec-driven application: does the implementation match the specs? — Specific suggestions: at least 2 actionable recommendations per review - File as GitHub Issues labeled “architecture-review” on their repos - This develops the “mentor engineers on architecture” capability from the JD
--	--

12:00	Lunch	60 min
	Form study groups for Module 5 pre-work	

13:00	Week 4 Retrospective & Development Map Update	60 min
	- Sailboat retrospective (recurring format): — Wind: What worked? (often: ADRs making trade-offs explicit, NanoClaw container experience) — Anchor: What was hard? (often: distributed system design, resisting urge to code before specs) — Rocks: Concerns for Modules 5–10? — Island: What’s exciting ahead? - EXERCISE 18: Personal Development Map v4 — Re-rate on five mindsets: how has Systems Thinking evolved? How about Precision? — Competency layers: Module 4 targeted Layer 2 (Architect) directly — Specific evidence: which ADR represents your best architectural thinking? — Compare Day 1 vs. now: the shift from implicit decisions to explicit trade-off analysis - Instructor 1:1s through the afternoon	

14:00	Impossible Stuff Day: Self-Directed Exploration	105 min
	- Week 4 Impossible Stuff Day suggestions: — Redesign your Module 3 application using everything you learned this week — spec first, ADRs for every decision — Deep-dive into CRDTs: implement a simple CRDT (G-Counter or LWW-Register) from scratch — Explore ClawSwarm’s multi-agent architecture in preparation for Module 6 — Write an in-depth blog post: “What I learned about system design in one week” — Read DDIA Chapter 5 (Replication) — the hardest chapter, worth the investment — Design a system for a problem from your previous job using the 5-step framework and trade-off matrix — Experiment: ask Claude Code to write ADRs for you, then critique the quality - Share explorations in last 15 min	

15:45	Break	15 min
--------------	--------------	--------

	• Stretch
--	---

16:00	Badge Ceremony & Week 4 Close	30 min
	• Badge ceremony: Systems Architect badge for students who completed all deliverables • Achievements: strongest ADR, best security analysis, most insightful trade-off matrix, best architecture presentation • Read the Developer Manifesto together (recurring ritual) • Midpoint marker: you're 40% through. You can now build fast (M3) AND design well (M4). Next: making it reliable (M5), then rapid prototyping under pressure (M6). • Module 5 preview: Data, Integration & Reliability — SLOs/SLIs, chaos engineering, data pipelines, incident response	

Instructor Notes — Day 5: The architecture review presentations are the week's culminating assessment and the first time students present at "architecture review board" formality. The 7-minute format is tighter than Module 3's 5-minute demo because it requires covering more conceptual ground. Coach students who present only their code (no trade-offs, no ADRs) to restructure — the presentation IS the assessment of architectural thinking, not implementation skill. The written reviews (Exercise 17) are where students practice the "mentor engineers on architecture" skill. Watch for reviews that are either too gentle (no real feedback) or too harsh (attacking without suggesting alternatives). The best reviews are specific, actionable, and empathetic.

Appendix A: Exercise Summary

#	Day	Exercise	Type
1	1	Write ADR for an implicit Module 3 decision (context, decision, consequences, alternatives)	Documentation
2	1	System design Problem 1: real-time notification system (single-service)	Design
3	1	System design Problem 2: e-commerce order pipeline (multi-service)	Design
4	1	Start design journal: commit designs, ADRs, and trade-off reflections	Documentation
5	2	System design Problem 3: collaborative document editor (distributed)	Design
6	2	Build a trade-off analysis matrix and apply to Problem 3 options	Analysis
7	2	AI-assisted system design: generate, evaluate, and critique AI architecture output	Analysis
8	2	Use Kiro to generate full spec for one design problem (enhanced with EARS notation)	Spec
9	3	Read and analyze NanoClaw codebase; compare architecture with Nanobot and OpenClaw	Analysis
10	3	Deploy NanoClaw in containers; test isolation, resource limits, multi-instance	Hands-on
11	3	Security architecture analysis: attack surface, assessment, recommendations report	Security
12	3	ADR for container isolation decision (4 options compared with trade-off matrix)	Documentation
13	4	Kiro spec generation and peer review gate: requirements, design, tasks (no code yet)	Spec
14	4	Spec-driven build: implement application following tasks.md with Agent Hooks	Build
15	4	Comparative analysis: spec-driven vs. build-first development approaches	Analysis
16	5	Architecture review presentation: 7-min portfolio presentation + peer Q&A	Presentation
17	5	Written architecture reviews for 2 peers (filed as GitHub Issues)	Collaborative
18	5	Personal Development Map v4 update (builder → architect growth evidence)	Reflection

Appendix B: Deliverables Checklist

Complete all items to earn the Systems Architect badge:

1. **System Design Journal** — 3 system designs of progressive complexity (notification system, order pipeline, collaborative editor) with diagrams and trade-off analysis, committed to design-journal/
2. **Architecture Decision Records** — Minimum 4 ADRs: (1) Module 3 retrospective ADR, (2) notification or order pipeline ADR, (3) distributed system conflict resolution ADR, (4) container isolation ADR. All using Context → Decision → Consequences format with alternatives considered
3. **Trade-Off Analysis Matrix** — Reusable matrix template with weighted scoring, applied to at least 2 architectural decisions, committed to portfolio repo
4. **NanoClaw Security Analysis** — Structured report covering: attack surface mapping, architecture-level assessment, comparison with Nanobot/OpenClaw, 5–10 prioritized recommendations with severity ratings
5. **NanoClaw Deployment** — Working NanoClaw instance in containers with verified isolation (filesystem, network, resource limits) and multi-instance demonstration
6. **Spec-Driven Application** — Application built entirely through Kiro's workflow: requirements.md, design.md, tasks.md committed BEFORE implementation code, with peer review gate evidence
7. **Spec-Driven vs. Build-First Comparison** — Structured analysis comparing Module 3 and Module 4 approaches across 6 dimensions (speed, quality, completeness, changeability, team scalability, AI effectiveness)
8. **Architecture Review Contributions** — 7-minute presentation delivered, 2 written architecture reviews for peers filed as GitHub Issues with actionable recommendations
9. **Personal Development Map v4** — Updated self-assessment with evidence of growth from Technical Foundation (Layer 1) to Architect (Layer 2) thinking

End of Module 4 Lesson Plan